

Vyomi — Production Deployment Guide

This guide covers production-mode deployment of the Vyomi simulator. For local dev, use `docker compose up` and skip this doc.

Differences from dev mode

Concern	Dev mode	Production mode
Vault	<code>-dev</code> flag, in-memory unseal, root token in env	file backend, persistent volume, init/unseal script
Restart	All Vault keys lost	Keys persist across restarts
Backups	None	Volume-backed; back up <code>vault-data/</code>
Multi-tenant	Single instance	Single instance (multi-instance not supported in v1.0)
Rate-limit	Per-tenant token bucket (Free=10rps, Enterprise= ∞)	Same (already production-grade)
SSO	OIDC validation against IdP JWKS	Same
Helm chart	Generated on-the-fly	Generated on-the-fly + tested via <code>kubectl apply</code>

Vault — switching from dev-mode to production

The default `docker-compose.yml` runs Vault in `-dev` mode for fast local iteration. For production, **you must switch to the file backend** or all encryption keys + secrets are lost on every restart.

Steps

1. Replace the `cloudlearn-vault` service block in `docker-compose.yml` with the production version below:

```
cloudlearn-vault:
  image: hashicorp/vault:1.15
  cap_add: [IPC_LOCK]
  ports: ["8200:8200"]
  volumes:
    - vault-data:/vault/data
    - vault-shared:/vault/shared
    - ./packaging/vault/vault-config.hcl:/vault/config/config.hcl:ro
    - ./packaging/vault/vault-init.sh:/usr/local/bin/vault-init.sh:ro
  command: >
```

```

sh -c "
  vault server -config=/vault/config/config.hcl &
  VAULT_ADDR=http://127.0.0.1:8200 sh /usr/local/bin/vault-init.sh
  wait
"

restart: unless-stopped

volumes:
  vault-data:
  vault-shared:

```

2. Update the `cloudlearn-simulator` service to read the root token from the shared volume (instead of env var):

```

cloudlearn-simulator:
  volumes:
    - vault-shared:/vault-shared:ro
  environment:
    CLOUDLEARN_VAULT_TOKEN_FILE: /vault-shared/root_token # NEW

```

And in `server.py` Vault client init, prefer the file when set:

```

token = os.environ.get("CLOUDLEARN_VAULT_TOKEN") or \
    open(os.environ.get("CLOUDLEARN_VAULT_TOKEN_FILE", "")).read().strip()

```

3. **Back up** `/vault/data/init.json` — it contains the root token + the single unseal key. If you lose it, all Vault data is unrecoverable. Copy to an offline secrets manager (1Password, HashiCorp Cloud, AWS Secrets Manager, etc.).
4. For HA, swap the storage block in `vault-config.hcl`:

```

storage "raft" {
  path      = "/vault/data"
  node_id   = "node1"
}

```

and add a second / third Vault instance.

Backups

```

# 1. Back up Vault data
docker compose exec cloudlearn-vault tar czf - /vault/data | \
  gpg -c > vault-backup-$(date +%Y%m%d).tar.gz.gpg

# 2. Back up the simulator state DB (SQLite)
docker compose cp cloudlearn-simulator:/app/.cloudlearn_state.sqlite3 \
  state-backup-$(date +%Y%m%d).sqlite3

```

```
# 3. Back up Postgres + MySQL (RDS / Cloud SQL surrogates)
docker compose exec cloudlearn-postgres pg_dumpall -U cloudlearn > pg-backup-$(date +%Y%m%d).sql
docker compose exec cloudlearn-mysql mysqldump --all-databases -ucloudlearn -p > mysql-backup-$(date +%Y%m%d).sql
```

Recommend running these daily via cron + retaining 14 days locally + 90 days to off-host storage (S3 / GCS).

Helm deployment (Enterprise tier)

The `/api/runtime/helm/chart.tar.gz` endpoint (Enterprise-only) returns a production-ready chart. Quick path:

```
# As Enterprise-tier user
curl -O https://your-cloudlearn-endpoint/api/runtime/helm/chart.tar.gz
helm install cloudlearn ./cloudlearn -n cloudlearn --create-namespace

# Air-gapped: download bundle, transfer, install
curl -O https://your-cloudlearn-endpoint/api/runtime/helm/airgap-bundle.tar.gz
# Transfer to air-gapped host
tar xzf cloudlearn-airgap.tar.gz
cd cloudlearn-airgap
bash install.sh
```

Rate-limiting

Active out of the box. Tier ceilings (`core/tier_policy.RATE_LIMITS_RPS`):

Tier	Sustained RPS	Burst
Free	10	40
Student	50	200
Developer	200	800
Enterprise	unlimited	unlimited

429 responses include `Retry-After` header + `code: "rate_limited"` body.

SSO setup (Enterprise tier)

1. Configure your IdP (Okta / Auth0 / Azure AD / Google Workspace) to issue tokens for a `cloudlearn` audience.
2. POST to `/api/runtime/sso/configure` :

```
{
  "idp_discovery_url": "https://your-tenant.okta.com/.well-known/openid-configuration",
}
```

```
"audience": "cloudlearn",
"user_mapping": "email"
}
```

3. Subsequent API calls with `Authorization: Bearer <jwt>` are validated against the IdP's JWKS. Fail = 401 `sso_invalid_token`. Pass = the user identifier is stashed on `request.state.sso_user`.

Observability — known gaps (deferred to v1.1)

- No Prometheus `/metrics` endpoint yet
- No structured JSON logging (stdlib `logging` only)
- No distributed tracing

Mitigation for v1.0: run behind a sidecar that scrapes `docker stats` and parses logs. Real metrics + tracing planned for v1.1.

Single-instance constraint

Vyomi v1.0 runs as a single instance. State is stored in:

- `.cloudlearn_state.sqlite3` (tenants, spaces, license, audit log)
- Vault `/vault/data/` (KMS keys + secrets)
- Postgres/MySQL persistent volumes (RDS/Cloud SQL surrogates)
- MinIO `/data/` (S3 bytes)

If you run two instances against the same volumes, you'll see race conditions

- inconsistent in-memory caches. Multi-instance coordination is a v1.1 roadmap item.

Health checks

- `/healthz` — simulator HTTP responsiveness
- Vault health: `curl http://localhost:8200/v1/sys/health`
- Backend reachability: `/api/runtime/backends` returns status for all 8 backends